AgentProf: Semantic Profiling for AI Agents

Anonymous submission

Abstract

AI agents increasingly orchestrate long-running, multi-step activities involving thousands to millions of interactions with users, tools, and system resources. To improve quality, safety, and cost efficiency of AI agents, developers need to determine where failures concentrate, which workflows trigger unsafe effects, and which task categories consume the most budget. To answer similar questions in traditional systems software, profiling attributes resource consumption to responsible entities, such as code paths, by aggregation. Yet existing agent observability tools focus on single-execution debugging and tracing rather than profiling, making these questions difficult to answer as trajectories grow long and complex. Agent behavior differs from code execution, creating two challenges for profiling: the responsible entities are semantic categories such as task intent and workflow phase, not code paths, and agent trajectories lack the stable identifiers and structural hierarchies that make attribution possible. We propose a *semantic operation stack model* that adapts profiling to agent trajectories. Uniform *operations* represent all agent activities, and *operation stacks* replace the runtime call stack, enabling hierarchical attribution at different levels of granularity from the same data. We observe that an agent's task occupies a contiguous span of a trajectory and decomposes into contiguous subtasks, so task structure is a set of nested named intervals recoverable by finding only the transitions between them, and we design *recursive operation segmentation*: it splits a trajectory where the agent's responsibility changes, names the intervals on both sides, and recurses to supply the missing stack. AgentProf implements this in a profiler that compiles local agent trajectories into pprof-compatible profiles. On real trajectories and public datasets, recursive segmentation agrees with human annotations at 0.764 B^3 F1, and profiles improve ranking over benchmark diagnostics by 0.031–0.117 MAP.

Introduction

AI agents (Yang et al. 2024; Anthropic 2025; Xie et al. 2024) orchestrate multi-step activities that span two layers, high-level intent (prompts, LLM calls, tool invocations) and low-level system effects (process spawns, file and network I/O). As agents evolve from short, scripted workflows into complex systems that run for hours and days, with a large number of interactions each, production teams accumulate many such trajectories over weeks or months.

To improve agent quality, safety, and cost efficiency, developers need to analyze operational behavior across trajectories and interactions. Which categories of tasks consume the most token budget? Where do failures concentrate across workflows? Which behavioral patterns trigger unsafe system effects? Answering these questions requires analysis and evaluation across many trajectories (Mazaheri and Mazaheri

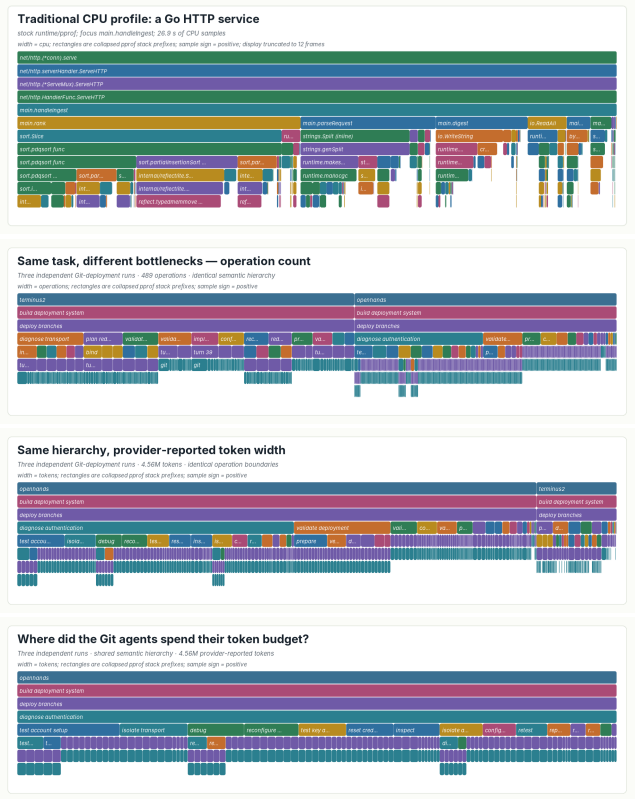

Figure 1: One renderer, four standard pprof profiles. *(a)* CPU time in a Go HTTP service. *(b, c)* Three agent executions of one Git-deployment task under one fixed operation hierarchy: width is operation count in (b) and tokens in (c). The hierarchy is identical in both; only the measure changes. *(d)* The same token profile focused on the shared `diagnose authentication` responsibility. Every panel is read the same way: width is aggregate cost, a parent contains its children, and identical stacks fold.

2026; Lù et al. 2025), a task becoming costly for manual inspection or per-trajectory LLM evaluation (Zheng et al. 2023) at scale. In traditional systems software, profiling answers these questions by attributing resource consumption to responsible entities by aggregation.

Yet existing agent tools support debugging and tracing but not profiling. Some (LangChain 2024; Langfuse 2024; Arize AI 2024a; OpenTelemetry 2024) organize events into per-execution span trees, effective for diagnosing a single run but unable to aggregate by semantic category. Others (Datadog 2025; Laminar 2025) cluster inputs or extract structured signals, but characterize *input distributions* ("30% of users ask code questions") rather than *resource attribution* ("review tasks consume 40% of the token budget"), because tags at the request level do not propagate to downstream tool calls and system effects.

Adapting profiling to agent trajectories is challenging. In traditional software, the responsible entities are code paths. Profiling is straightforward because function names are stable and runtime call nesting provides the attribution hierarchy. Agent behavior differs fundamentally. To answer the questions above, the responsible entities need to be semantic categories such as task intent and tool operation, not code paths. Agent trajectories lack the two properties that make traditional profiling possible. Prompts are natural language, so two prompts expressing the same intent share no common string. Agent events have no runtime call-stack hierarchy because prompts, tool calls, and system effects are not nested by execution the way function calls produce stack frames: a sequence of prompts can be a task or multiple tasks, a task can be part of a bigger task.

Despite these differences, the fundamental profiling methodology transfers to agent trajectories: project resources onto responsible entities, assign stable tags, and attribute hierarchically. We propose a *semantic operation stack model* with two components. *Operations* are uniform records with string fields and additive measures that represent all agent activities (prompts, LLM calls, tool invocations, and system effects) without type-specific objects. *Operation stacks* provide hierarchical attribution, replacing the runtime call stack. Choosing different fields changes the attribution granularity: the same operations can be attributed by task category, by execution phase, by session, or by action type, and one fixed hierarchy replays under any additive measure (Figure 1).

We implement this model in AgentProf, an offline profiler that compiles local agent trajectories into pprof-compatible profiles (Figure 2). One algorithm supplies the tags the trajectory does not carry, and its design follows from one observation about agent work: a task occupies a contiguous span of a trajectory and decomposes into contiguous subtasks, so task structure is exactly a set of nested named intervals, and recovering it requires only finding the transitions between them. This makes the problem segmentation rather than classification: no predefined taxonomy is required, and each branch's depth emerges from the content instead of a fixed schema. *Recursive operation segmentation* therefore reads a trajectory, cuts it at the points where the agent's responsibility changes, names the intervals on both sides, and recurses, so short action-first names (verb-led phrases like `diagnose authentication`) give agents the stable identifiers that traditional profiling gets for free from function names. The segmentation interface is implementation-neutral: an agent, a language model, or a statistical rule can all produce the same marks. Across all 405 released CodeTraceBench trajectories, recursive segmentation agrees with human stage annotations at 0.764 B^3 F1 (Bagga and Baldwin 1998), versus 0.663 for a statistical baseline and 0.541 for raw actions. On three complete localization benchmarks, the profile improves ranking over each benchmark's own diagnostic, raising MAP by 0.031, 0.107, and 0.117. As a navigation aid, it helps a reader reach equal ranking quality while opening 53.0% instead of 65.0% of the source evidence. One fixed hierarchy replays under operation-count and token measures with exact conservation, exposing bottlenecks that a count view understates by more than a factor of two.

This paper makes three contributions:

1. **Semantic operation stack model.** We identify the missing profiling layer in agent observability and propose a model of uniform operations and query-time operation stacks (Background and Design).
2. **System: AgentProf.** A profiler that implements this model, producing pprof-compatible output.
3. **Evaluation.** We evaluate profiler output against independent ground-truth annotations that stay hidden from the profiler, on eight public benchmarks and three real trajectory populations.

Background and Motivation

LLMs and AI Agents

A typical agent trajectory interleaves two layers of activity. At the intent layer, the agent receives a user prompt, issues LLM calls, and invokes tools (code execution, web search, file editing). Each tool invocation triggers system effects at the lower layer: process spawns, file reads and writes, and network requests. A single trajectory may contain hundreds of such intent-effect cycles, and a production team accumulates many trajectories over time.

System Profiling

Profiling attributes resource consumption to responsible entities by aggregation, as distinct from single-execution debugging and tracing. In traditional software, the profiling pipeline works as follows: a profiler periodically samples execution state, attaches each sample to the current call stack, and merges samples with identical stacks. Flame graphs (Gregg 2011) and pprof (Google 2024b) call graphs visualize the result, where width or node size represents aggregate cost. Perfetto (Google 2024a) generalizes this with SQL-based analysis and derived events. These tools support flexible aggregation, but they all assume stable function names and a runtime call stack. Pprof's `tagroot`/`tagleaf` options can add label-derived pseudo-frames, but only on top of an existing execution stack that agent trajectories do not have.

A Motivating Case

Three independent agent attempts at one real Git-deployment task all failed to deliver the requested password-authenticated

endpoint. The developer's question: *what went wrong, and is it the same problem across all three runs?* Reading three transcripts totaling 489 operations and 4.6M tokens would take hours. Figure 1 places a conventional CPU profile beside the semantic profile of those three runs. Both are standard pprof profiles drawn by one renderer and read by identical rules: width is aggregate cost, a parent contains its children, and identical stacks recorded at different moments fold into one frame. What differs is where the frames come from. A call stack supplies `net/http.(*conn).serve` at every sample for free; nothing in an agent trajectory supplies `diagnose authentication`, because the three runs express that intent through different prompts and different shell commands. Once that name is constructed, the profile answers the engineer's question directly.

The profile answers in one view (Figure 1): all three runs share a `diagnose authentication` responsibility that consumes 46% of their combined token budget but only 21% of operation count. Drilldown shows all three executions verifying substitute transport paths without ever establishing the target endpoint. The insight: the agents kept retrying SSH variants instead of fixing the actual credential issue, and operation count alone would hide this because authentication commands are cheap to issue but expensive to verify.

A control experiment replays the same operations under native call trees and coarse action grouping: the authentication work scatters across 105 native calls and six action kinds (102 of 105 labeled merely `run`), but only semantic organization reunites it into one focusable cross-run path. This paper's evaluation returns to this case under RQ1 and asks whether the same behavior holds at population scale.

Challenges for Agent Profiling

Existing tools (LangChain 2024; Langfuse 2024; Arize AI 2024a; OpenTelemetry 2024) record agent events as per-execution span trees, effective for debugging a single run. AgentSight (Zheng et al. 2025) bridges intent and system-effect layers by correlating eBPF-captured system events with intercepted LLM traffic, but the resulting recordings still lack aggregate profiling.

Adapting profiling to agent trajectories faces two core challenges. The first is that the responsible entities differ. In traditional software, profiling attributes cost to code paths, but in agent trajectories the relevant entities are semantic categories such as task intent and workflow phase. Existing tools (OpenTelemetry 2024; Arize AI 2024b) do not capture these categories because prompts are natural language: two prompts and tool invocations expressing the same intent may be different, so the profiler cannot aggregate them the way it aggregates identical function names. The second challenge is that agent trajectories have no runtime hierarchy for attribution. In CPU profiling, the call stack determines attribution at every level: `malloc`'s cost belongs to `parse`, to `process`, and to `main` simultaneously. A sequence of prompts may constitute one task or several, and a task may be part of a larger workflow, but no runtime mechanism marks these boundaries or determines at which granularity to attribute cost. We quantify this in RQ1.

Design Requirements for Agent Profiling

Traditional profiling relies on three mechanisms that the call stack provides automatically. Agent profiling needs all three but must achieve them without a call stack:

R1: Cross-layer resource projection. In traditional profiling, each sample is taken inside a call context, so resource consumption is automatically attributed to the code path that caused it. Agent activities span two layers: intent (prompts, LLM calls) and system effects (file I/O, process spawns). The profiler must connect system-layer resource consumption to intent-level semantic categories.

R2: Stable tags for folding. In traditional profiling, function names are stable identifiers that enable folding identical stacks. Agent prompts are natural language, so the profiler must derive short, stable, repeatable tags before folding.

R3: Hierarchical attribution. In traditional profiling, the runtime call stack provides the attribution hierarchy: function calls nest, and folding identical stacks produces the tree that profiles visualize. Agent events are not nested by execution, so the profiler must construct attribution hierarchies from the data.

Design

The three requirements (cross-layer resource projection, stable tags, and hierarchical attribution) drive the design. Operations satisfy R1 by representing intent and system-effect activities in a single record with tag inheritance, so intent tags propagate to the system effects they trigger. *Recursive operation segmentation* satisfies R2 and R3 together: it derives short, stable interval names from trajectory content and nests those intervals into the attribution hierarchy that replaces the runtime call stack. *Operation stacks* then attribute resources at every level of that hierarchy at query time, so the same data supports multiple views without rebuilding the input. The profiling pipeline has four stages: (1) parse raw trajectories into operations, (2) segment trajectories into named nested intervals (or apply mapping rules), (3) project each operation onto a stack frame sequence chosen at query time, and (4) fold operations with identical stacks, summing their weights.

Semantic Operation Stack Model

Because agent activities span both intent and system-effect layers, a profiler should not distinguish between them in the data representation. AgentProf therefore represents every activity as a single abstraction, the `operation`, a weighted record with string fields and additive measures. Each operation carries string fields (`project`, `agent`, `session`, `prompt_tag`, `kind`, `model`, `path`, `domain`, `status`, ...) and additive measures (token count, duration, event count). A prompt, an LLM call, a tool invocation, a file read, a GUI click, and a process event are all operations with the same schema, distinguished only by which fields carry values.

An `operation stack` provides hierarchical attribution for agent trajectories, replacing the runtime call stack.

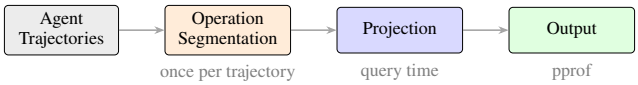

Figure 2: Data-flow and algorithm pipeline. Local agent histories enter through `agent-session` parsing; public datasets enter as operation JSONL. Both paths produce uniform operations. Recursive operation segmentation, projection, and folding are algorithms (segmentation runs once per trajectory; projection and folding run at query time) applied identically to both sources.

In CPU profiling, the call stack determines which code paths share responsibility for a resource cost. An operation stack serves the same role: an ordered list of fields $[f_1, f_2, \ldots, f_k]$ projects each operation o to a frame sequence $\langle o.f_1, o.f_2, \ldots, o.f_k \rangle$, and operations with identical sequences are merged, summing their weights. Unlike a call stack, the fields are chosen at query time, not determined by execution. The same operations can therefore be attributed as a flat task summary, a per-session tree, a semantic phase/action hierarchy, or a trajectory using the dataset's own annotations, and changing the fields changes the attribution without changing the data. Sessions and spans are optional operation fields, not fixed hierarchy levels, so the same data supports both debugging views (include `session`) and aggregate profiling views (exclude it). The frame sequence need not come from fields alone: recursive segmentation supplies each operation's covering interval-name chain as a variable-depth frame sequence, and field lists and interval paths compose in one stack. A view is a triple (φ, σ, w): a predicate φ selects which operations participate, a stack function $\sigma = [f_1, \ldots, f_k]$ maps each operation to its frame sequence, and a weight function w assigns a nonnegative measure. Four built-in views cover common questions: `tokens` (LLM calls weighted by token count), `time` (all timed operations weighted by duration), `files` (file operations weighted by event count), and `network` (network operations weighted by event count).

Recursive Operation Segmentation

The algorithm rests on one structural intuition: an agent's task occupies a contiguous span of the trajectory, and it decomposes into smaller contiguous subtasks. Task structure is therefore exactly a set of nested named intervals, and recovering it only requires finding the transition points between them.

Because agent work has no predefined taxonomy but transitions are visible in the content, the core algorithm is segmentation, not labeling. Instead of assigning each step a class, the algorithm reads a trajectory and finds the transition points where the agent's responsibility changes. At each transition it cuts the trajectory and names the intervals on both sides; it then recurses into each interval, cutting again wherever a finer responsibility change remains, and stops when an interval reads as one coherent piece of work. Formally, a trajectory is an ordered step sequence $T = (t_1, \ldots, t_n)$, and a segmentation is a set S of named intervals over T that is nested (any two intervals are disjoint or one contains the other), covers every step, and contains the whole-session interval as its root. The algorithm computes S by one recursive rule: $\textsc{Segment}(I)$ selects zero or more transition points inside interval I, which partition I into consecutive child intervals; each child receives a short action-first name and is segmented recursively; selecting no transition terminates the branch. A step's operation path is the name chain of the intervals containing it, from the session root to its innermost interval; equal paths fold across sessions. Segmentation is the right primitive for three reasons. First, transitions are what a trajectory actually exhibits: agent work has no predefined taxonomy, but a shift in what the agent is trying to do is visible in the source content itself. Second, cuts compose: nested intervals form a hierarchy whose depth emerges per branch from the content rather than from a prescribed schema. Third, naming intervals rather than steps yields identifiers at the granularity where recurrence lives, so the same short action-first name (one to three words) reappears across sessions and folds. Concretely, in one Git-deployment execution the algorithm cuts the session into `build deployment system` and, inside it, cuts again where the agent stops writing configuration and starts debugging access, yielding `deploy branches` and `diagnose authentication`; a step inside the latter carries the path `build deployment system > diagnose authentication`, and because the same names reappear in the other two executions, their costs fold together in Figure 1. Any implementation able to find transitions and name intervals (an agent, a language model, or a statistical rule) can serve as the segmenter. In evaluation, we use Codex (OpenAI 2025), a frontier code agent, invoked once per trajectory with one fixed instruction:

> *Input:* one trajectory (each step's prompt, command, and output summary, with no labels or scores).
> *Action:* read it, emit a mark wherever the responsibility changes, and use AgentProf to check and update the segmentation, and re-read and update the marks iteratively until you think the segmentation is complete.
> *Output:* sparse path marks, e.g.,
> `step 1: [deploy git server]`
> `step 4: [deploy git server, diagnose SSH access]`
> `step 15: [deploy git server, verify web endpoint]`
> —one line only where the path changes; names of one to three action-first words.

The sparse marks still produce a complete segmentation: steps between two marks inherit the preceding path. Segmentation is not a one-shot labeling pass: the agent can revise marks, refine interval names, and add deeper cuts iteratively until it judges the hierarchy complete. Revisions are incremental: changing one interval does not require re-annotating the entire trajectory. Although we evaluate segmentation offline, the model also supports online incremental annotation during agent execution.

Implementation

AgentProf is implemented as a Rust CLI (∼9.8K LOC, Figure 2). It reads Codex and Claude Code local JSONL history

files and AgentSight (Zheng et al. 2025) recordings for system effects. The pipeline has two stages: first, preprocessing extracts a readable trajectory summary; then, the segmentation agent reads this summary and writes interval marks to an annotation file, which it can revise until satisfied. The CLI validates that marks are nested and cover every step, then emits a standard pprof protobuf profile that `go tool pprof` reads directly. Projection is deterministic: each operation contributes its semantic path and LLM/tool evidence, with session identities kept as pprof labels rather than visible frames, so equal semantic prefixes fold across sessions. Switching the additive measure replays the same annotations and changes only stack widths.

Non-LLM statistical segmenter. The statistical segmenter scores adjacent action transitions by normalized pointwise mutual information (Bouma 2009) and calibrates a label-free cutoff with occurrence-weighted two-means (MacQueen 1967), treating detailed recurrence as continuity that can remove but never add a coarse boundary. An optional reference-calibrated mode fits one scalar cutoff on disjoint reference groups.

Evaluation

The evaluation addresses four research questions. **RQ1:** Does semantic profiling improve resource attribution? **RQ2:** Does profiler output correspond to real problems? **RQ3:** How accurate are the tags? **RQ4:** What is the profiling cost? Together they form one chain: semantic responsibility can be recovered accurately (RQ3), the recovered hierarchy aggregates across runs while conserving every additive measure (RQ1), the aggregate corresponds to real problems and reduces the evidence a reader must open (RQ2), and construction and replay costs are measured and practical (RQ4).

We evaluate three data classes. *Real inputs*: 41 long-horizon coding-agent sessions (three of which repeat one Git-deployment task and form the RQ1 case), 440 mixed-outcome web-agent trajectories (Lù et al. 2025), and 42 development sessions from the authors' workstation. *Annotated trajectories*: 405 CodeTraceBench trajectories with 20,866 operations and 2,948 human-verified stages (Li et al. 2026), 287 OSWorld-Human sessions (WukLab 2025), 1,012 AgentBoard goals (Ma et al. 2024), and 2,737 published action labels (Bouzenia and Pradel 2025). *Problem-localization benchmarks*: the complete AgentProcessBench, HINTBench, and TraceElephant workloads, where every trajectory carries an independently annotated faulty step, over 27,346 operations (Fan et al. 2026; Wang et al. 2026b; Chen et al. 2026). All annotations, outcomes, and scores stay hidden from every method until its output is frozen. Mean operations per trajectory are 8.5 (AgentProcessBench), 13.9 (OSWorld-Human), 24.0 (HINTBench), 27.1 (TraceElephant), and 51.5 (CodeTraceBench), so the benchmark trajectories are short-to-medium horizon while the 42-session workstation population—whose longest sessions span tens of hours—supplies the long-horizon regime; the RQ4 union covers 27,765 operations.

RQ1: Does Semantic Profiling Improve Resource Attribution?

Setup. We test one necessary consequence of the claim: whether a fixed semantic hierarchy reveals materially different bottlenecks when the additive resource measure changes. The population contains 41 real long-horizon agent trajectories and 5,750 operations, including three independent executions of one Git-deployment task with 96 recursive annotations. The resulting hierarchy reaches semantic depth five without a fixed limit.

The motivating case revisited. The Git-deployment case from Section established that semantic organization reunites cross-run responsibility that native and coarse organizations scatter. Here we add a third measure: elapsed time. The same hierarchy replays under all three measures with exact conservation, and the `diagnose authentication` subtree's share rises from 21% (count) through 37% (time) to 46% (tokens)—changing only the measure moves the attributed share by more than a factor of two. Separately, real AgentSight (Zheng et al. 2025) eBPF recordings replay the same way: the R114 population folds all 1,520 process and file effects under task responsibilities through the recorded wrapper tool with exact conservation, demonstrating the system-effect layer end to end.

Use case 2: where did this project's agent budget go? A developer spent weeks building AgentProf and this paper with coding agents, accumulating 42 sessions (18 Codex, 24 Claude Code), 1,252 prompts, 5,620 LLM calls, and 1.38 billion tokens. The question: *what did the agents actually spend tokens doing?* The profile (Figure 3) shows the answer is not one dominant sink but a broad distribution: the largest path (`refine paper > align evaluation`) holds only 1.7% of tokens. The three longest sessions (spanning tens of hours each) keep distinct dominant responsibilities (evaluation alignment, evidence inspection, and merge resolution), so session structure is preserved rather than averaged away. Switching to operation count shifts where mass concentrates: token mass stays at prompt depth (70%), while operation mass resolves deeper (44% at depths three and four) exactly where repeated engineering work concentrates.

The same hierarchy replays under FILE-READ, FILE-WRITE, and NETWORK-target widths with exact conservation of 737, 31, and 61 target references (Figure 4); in the FILE-WRITE view, each created file appears as an exact drilldown leaf beneath the operation that created it. These views ground the Introduction's system-effect question at the substrate level: every recorded file and network target is attributed to the semantic responsibility that touched it, making side effects auditable per responsibility.

Takeaway. One hierarchy attributes multiple resources with exact conservation; switching measures reveals bottlenecks that operation count alone would hide.

RQ2: Does Profiler Output Correspond to Real Problems?

RQ2 asks whether target-blind (the profiler cannot see the ground-truth annotations) profiler output corresponds to in-

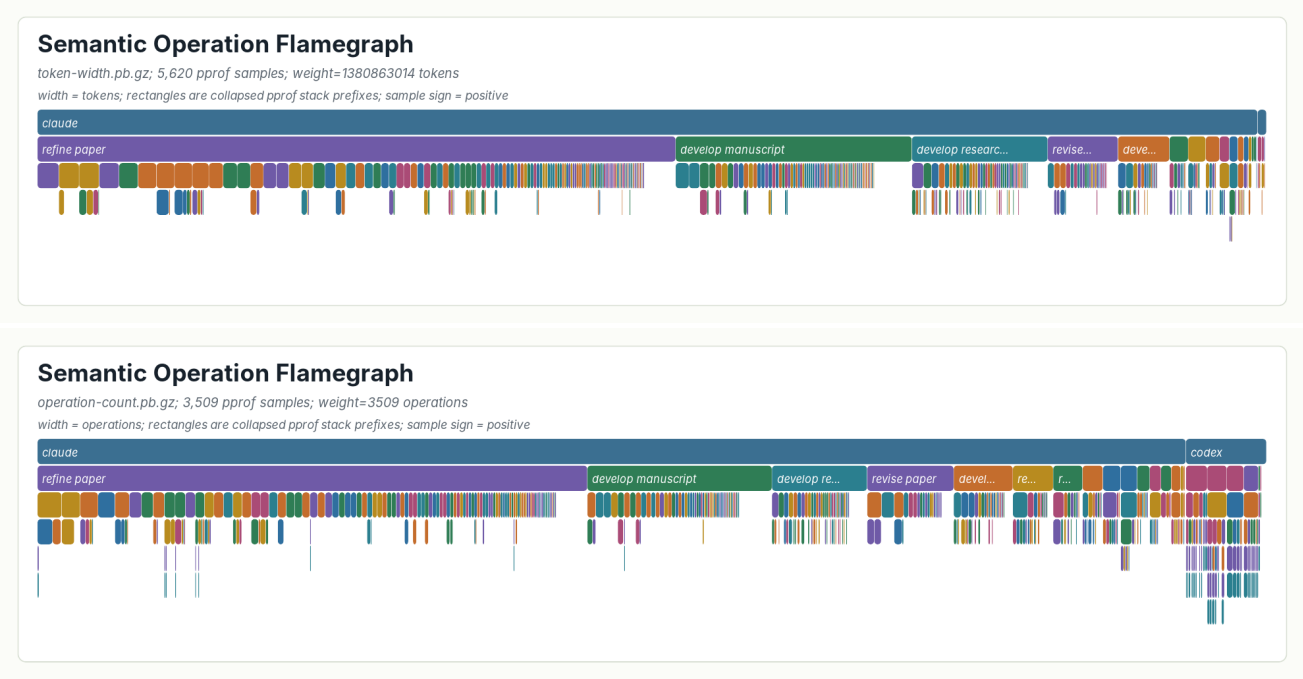

Figure 3: The 42-session population under one fixed semantic hierarchy. Width is tokens (top) or operation count (bottom); second-level frames are annotated responsibilities (`refine paper`, `develop manuscript`, ...), with deeper refinements available for drilldown in stock pprof.

dependently annotated real problems. We answer at three levels: a population-scale differential case whose recovery signal matches expert looping labels, a supplement test over three complete localization benchmarks, and a reading-index study measuring how much source evidence a reader must open.

Use case 3: what do failing runs do differently? A team has 440 web-agent runs (Lù et al. 2025) over 125 tasks, where every task has both successful and failed attempts (202 successful, 238 failed). The question: *what behavior separates success from failure?* Reading 440 transcripts (7,229 operations) is infeasible. A signed difference profile (bad minus good, Figure 5) answers at a glance: failed runs spend 45% of their steps in `recover interaction` (retrying, re-searching, re-navigating) versus only 12% for successful runs. Drilldown decomposes the recovery responsibility into verification challenges, repeated searches, mistaken navigation, and record retries, with source labels identifying which sessions contributed each width. The insight: failing agents get stuck in retry loops rather than backing out and trying a different approach. This structure matches independent expert looping labels at AP 0.63 (baseline 0.40), confirming the profile surfaces a real behavioral pattern, not an artifact of step count.

Supplementing benchmark-native diagnostics. *Setup.* Each benchmark trajectory has one annotated faulty step; a method outputs a ranking of that trajectory's operations, scored by average precision (AP approximates the reciprocal rank of the faulty step) and averaged as MAP (Robertson 2008). We run the complete AgentProcessBench, HINTBench (the complete released test snapshot, 536 of the paper-reported 629 trajectories), and TraceElephant workloads; operations, target-blind paths, benchmark predictions, and scoring are fixed before test targets are loaded. Each query is one trajectory containing an annotated target, and we report non-interpolated per-query AP averaged over 614, 400, and 220 target-bearing queries; all 522 zero-positive trajectories are consumed for population coverage but excluded from MAP because AP is undefined without a relevant item. The *direct diagnostic* is the per-operation output of a benchmark-native process judge (AgentProcessBench risk units) or trajectory localizer (HINTBench, TraceElephant), which for TraceElephant reads the full trace and reference answer before naming the responsible agent and decisive step. *Direct-only* ranks by that diagnostic alone; *Direct+AgentProf* ranks lexicographically by (direct diagnostic, target-blind group score) and can therefore only refine exact diagnostic-score ties; *Direct+AgentProf-Raw* is the information-matched ablation of AgentProf that groups by raw action instead of semantic names, with identical source-kind, tool/call, and outcome suffix and aggregation; *AgentProf-only* drops the benchmark diagnostic (Table 1).

Results. Direct+AgentProf beats Direct-only by 0.031 [0.024, 0.039], 0.107 [0.093, 0.120], and 0.117 [0.088, 0.148], using 10,000 paired resamples of trajectory clusters within benchmark-defined strata, so every paired interval lies above zero (Figure 6). The semantic and AgentProf-Raw refinements reach the same ranking quality (.893, .518, and .324), confirming that grouping plus retained evidence—not

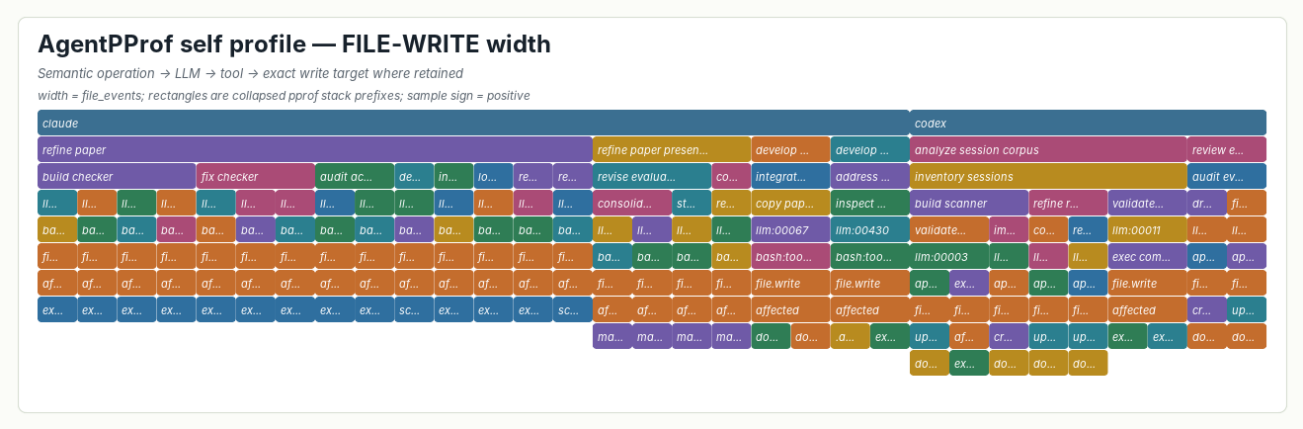

Figure 4: FILE-WRITE view of the same hierarchy: width is write count, with each created file preserved as a drilldown leaf.

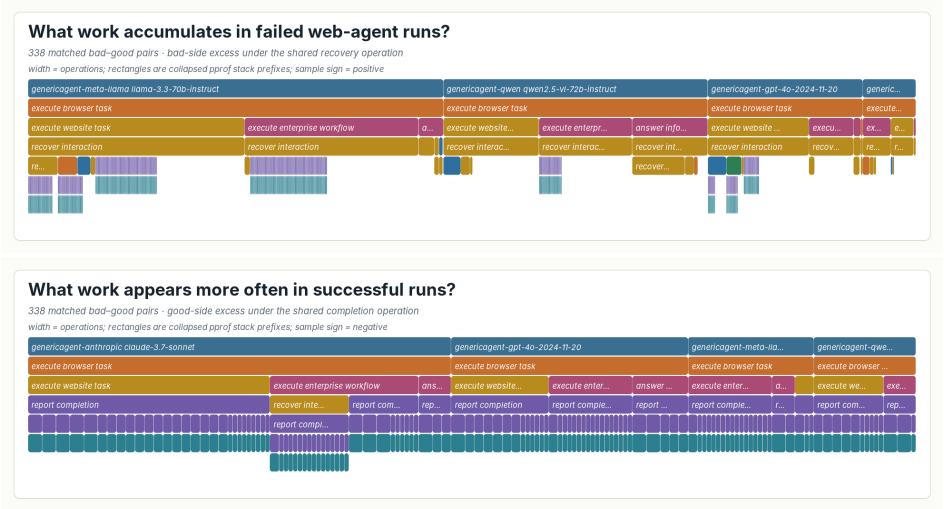

Figure 5: One signed profile over all 338 bad–good pairs, read from both sides. The signed profile is the difference of two nonnegative profiles, bad minus good, which stock pprof renders as positive and negative sample values. *Top:* positive samples under `recover interaction`, 3,286 bad-side versus 455 good-side occurrences. *Bottom:* negative samples under `report completion`, 135 bad-side versus 191 good-side occurrences. These are aggregate diagnostic differences, not causal effects.

naming alone—drives the localization gain over Direct-only. The semantic prefix earns its value in two other measured roles: cross-run attribution (RQ1) and directing a reader's attention, which the following study measures.

Profile-guided reading on TraceElephant. On the complete 220 target-bearing queries, a fixed external Grok-family CLI reader receives target-blind packets—task text, operation IDs, and source-visible content—with unranked operations appended in original order deterministically and one single-shot call per stage. Full-trace reading reaches MAP .502 at a mean 12,615 input tokens per query, versus .209 for the benchmark's Direct-only diagnostic and .326 for Direct+AgentProf. A two-stage profile-guided variant first shows only the semantic operation skeleton, with no source content, and the reader selects at most five groups; stage two then opens only those groups' evidence. It reaches MAP .455 while opening 53.0% of the source content, and its stage-one selections never fell back to a default: the five-group budget saturated on 99.5% of queries, re-parsing the uncapped stage-one responses shows the reader proposed more than five groups on zero of 220 queries, and every missed target group was entirely absent from its ordered selection rather than cut off by the budget. Under the information-matched AgentProf-Raw skeleton, ranking quality is statistically unchanged (MAP .465; paired delta $+.010$ $[-.021, +.042]$), but the reader opens significantly more content: 65.0% versus 53.0%, paired delta $+.120$ $[+.103, +.137]$, and 2.80 [1.96, 3.60] more evidence operations. Semantic naming's measured contribution in this regime is therefore attention concentration at equal quality. A per-query full read is also bounded by the model context window (the 4,558,192-token Git population cannot be read whole), whereas skeleton-guided reading stays available at any trace length once the

Workload	Direct+ AgentProf	Direct+ AgentProf-Raw	Direct only	AgentProf only
AgentProcessBench	**.894**	**.893**	.863	.791
HINTBench	**.517**	**.518**	.411	.432
TraceElephant	**.326**	**.324**	.209	.259

Table 1: MAP over the complete RQ2 populations (614, 400, and 220 target-bearing queries). AgentProf-Raw ablates only AgentProf's semantic prefix; bold marks each workload's best methods, and the two direct-first columns are statistically indistinguishable on every workload. Higher is better.

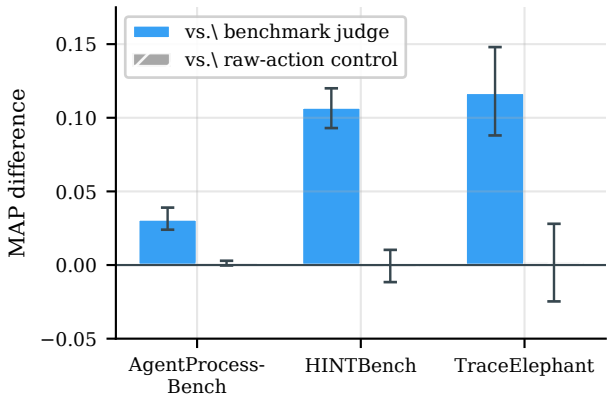

Figure 6: Paired MAP differences over the complete RQ2 populations, with 95% intervals from 10,000 trajectory-cluster resamples of unrounded per-query APs. Adding the target-blind profile to each benchmark's own judge improves ranking on all three workloads (left bars, intervals entirely above zero), whereas the semantic prefix and the information-matched AgentProf-Raw ablation are indistinguishable (right bars, intervals spanning zero). Grouping plus retained evidence, not the semantic names, drives this gain.

profile exists. The reader is query-specific while the hierarchy is constructed once and replayed across queries and measures. *Takeaway.* Failure structure surfaced by the profile matches independent human labels at collection scale; profiles add localization signal on top of existing judges; and the semantic names buy evidence efficiency rather than ranking accuracy.

RQ3: How Accurate Are the Tags?

RQ3 asks whether recursive segmentation recovers task structure that humans would recognize.

Setup. Gold structure is the 2,948 human-verified stages over all 405 CodeTraceBench trajectories. B^3 F1 (Bagga and Baldwin 1998) asks whether operations that belong together end up grouped together; exact adjacent-boundary F1 (Ruokolainen et al. 2016) asks whether cuts land exactly where humans cut. Baselines receive identical inputs (Table 2). Each method output is evaluated at the level it predicts: literal names by accuracy and macro-F1 (Lewis et al. 2004),

Method	B^3 P	B^3 R	B^3 F1	Bound. F1
Direct Agent segmentation	0.793	0.736	**0.764**	**0.480**
Multi-resolution recurrence	0.782	0.575	0.663	0.266
Native source tree	0.975	0.249	0.397	0.259
Source-native step	0.983	0.221	0.361	0.246
Raw-action grouping	0.891	0.388	0.541	—

Table 2: Agreement with human stages on all 405 CodeTraceBench trajectories.

permutation-invariant partitions by V-measure (Rosenberg and Hirschberg 2007) or ordinary B^3, and adjacent interval boundaries by exact precision, recall, and F1; for legacy field-valued datasets a conversion step maps each predicted group into the same interval-annotation format before AgentProf folds the original additive weights. Direct Agents read complete source-only packets once and emit sparse complete-path marks over 17,148 source-native steps without access to official stages or outcomes before materialization: 4,496 marks at semantic depth one (3), two (2,873), three (1,588), or four (32), with no prompt requesting a depth. The adopted marks' name replay maps 3,895 open-vocabulary operation IDs to 783 stable action-first canonical IDs, re-expanding all operations from the unchanged marks, preserving the temporal occurrence and adjacent-boundary vectors exactly, and leaving no adjacent display-path collision; nonadjacent and cross-session equal names intentionally merge so recurring work folds in pprof.

Results. Direct segmentation reaches 0.764 B^3 F1 and 0.480 boundary F1, beating the strongest automatic baseline by +0.101 [0.087, 0.116] and +0.214 [0.193, 0.235] (paired task-cluster intervals). The marks conserve exactly 20,866 operations and 494,862,929 tokens. Predicted groups remain largely pure subsets of gold stages (B^3 precision 0.793): when segmentation disagrees with human stages, it subdivides work rather than merging unrelated responsibilities, preserving per-group coherence for downstream drilldown. The interface generalizes across method families: on 287 OSWorld-Human sessions a supervised boundary predictor (McCallum and Nigam 1998) reaches 0.739 boundary F1 / 0.816 B^3 F1 and the no-LLM recurrence segmenter 0.680 / 0.786 (strongest control: 0.645 / 0.678), and a target-blind TF-IDF/K-Means method reaches V-measure 0.557 on nine Mind2Web sessions and 0.815 on 100 ScienceWorld sessions (Deng et al. 2023; Wang et al. 2022). For closed-label literal tags, a fixed evaluation-only Qwen3.6-27B (Qwen Team 2026) classifier labels all 1,012 AgentBoard goals (Ma et al. 2024) into nine task families at 0.695 macro-F1 and 0.733 accuracy (majority baseline 0.044 / 0.248), and a separate classifier classifies all 2,737 published action labels from 120 AutoCodeRover, OpenHands, and RepairAgent trajectories (Sajadi et al. 2026; Bouzenia and Pradel 2025) at 0.498 macro-F1 and 0.628 accuracy (majority baseline 0.061 / 0.323). *Takeaway.* Recursive segmentation is the most accurate tested automatic constructor on both structural axes, and non-LLM methods implementing the same interface remain viable when no model is available.

RQ4: What Is the Profiling Cost?

RQ4 asks whether the profiling cost is practical: how much time and tokens does segmentation consume, and how fast is replay?

Setup. Cost separates into a one-time automatic segmentation pass per corpus and deterministic construction/replay afterwards. The profiling core (parsing, stack construction, folding, and serialization) is measured by comparing the fixed semantic hierarchy against the raw hierarchy on identical inputs over three runs across four complete public workloads and their union, on a 24-core workstation. This microbenchmark exercises the generic parse/fold/serialize core on public field-stack workloads; the annotation-mark representation's deterministic pipeline is timed separately. *Results.* Segmenting all 405 CodeTraceBench trajectories consumes 12,050,384 input and 231,886 output tokens in 37 minutes of model time. After annotation, the deterministic downstream pipeline completes in under 12 seconds. A fully instrumented end-to-end pass on the complete 440-session population completes all 12 outcome-blind batches in 3,521.6 s on a fixed two-worker schedule (58.7 minutes; summed worker time 6,661.7 s), consuming 12,039,417 actual input tokens (10,929,408 reported cached, 90.8%) and 312,433 output tokens—27,362 input and 710 output tokens per session—while one fresh complete pass over the three-session Git population takes 466.9 s and 832,544 actual input tokens. With marks fixed, constructing a 27,765-operation union profile takes 1.2 seconds at under 500 MiB peak memory, about 20% more time than raw-action grouping. Switching the additive measure is a replay at the same cost. Deterministic materialization of the full 440-session population takes 0.26 s for operations and 0.25 s for tokens, so construction cost is dominated by the segmentation step while materialization stays sub-second. *Takeaway.* A corpus pays minutes of model time once; every later view, measure switch, and drilldown costs about a second.

Related Work

Classic profilers fold runtime call stacks over stable code identity, and Pivot Tracing groups causally related measurements (Google 2024b; Gregg 2011; Mace, Roelke, and Fonseca 2015). Agent observability platforms and standards record spans for single-execution debugging (LangChain 2024; Langfuse 2024; Arize AI 2024a; OpenTelemetry 2024; Arize AI 2024b; Dong, Lu, and Zhu 2024; Balusu 2026; Wang et al. 2026d), and analytics layers roll up cross-trace hierarchies, workflow profiles, or input distributions (Datadog 2026; LangChain 2026; NVIDIA 2026; OpenTelemetry 2026; Datadog 2025; Laminar 2025), but none constructs recursive semantic responsibility with conserved additive measures and retained call evidence in a standard profile. Cross-run analyses (TraceProbe, Graphectory, Act·onomy, CHIEF, Hodoscope, TraceGraph) (Shu et al. 2026; Liu et al. 2026a; Gao et al. 2026; Wang et al. 2026c; Zhong, Saxena, and Raghunathan 2026; Nian et al. 2026) and per-run diagnosis and localization (Barke et al. 2026; Wang et al. 2026a; Mazaheri and Mazaheri 2026; Liu et al. 2026b; Mulian et al. 2026; Li et al. 2026; Fan et al. 2026; Wang et al. 2026b; Chen et al. 2026; In et al. 2026) answer what an agent did and which step went wrong. AgentProf instead asks how much of an additive resource a semantic category is responsible for: it recovers variable-depth responsibility from source content (where Act·onomy fixes three levels and TraceProbe one), conserves measures exactly across count, token, and time widths, keeps LLM/tool calls as drilldown leaves, and emits standard pprof—no compared system provides all four—while staying complementary to per-run diagnosis, as RQ2 measures directly.

Conclusion

Agent observability needs profiling, not only debugging. AgentProf shows that the semantic operation stack model, driven by recursive operation segmentation, brings hierarchical attribution to agent trajectories. Against independent annotations, segmentation recovers human stage structure at 0.764 B^3 F1, profiles add localization signal on three complete public benchmarks, and one hierarchy replays across additive measures with exact conservation. A strong reader that re-reads a whole trajectory per question remains the best single-query localizer, confirming that profilers and debuggers are complementary: the profile answers population questions and guides the reader to the evidence worth opening. Multi-project evaluation and tracing-ecosystem integration are the most impactful next steps.

References

Anthropic. 2025. Claude Code: An Agentic Coding Tool. https://code.claude.com/docs/en/overview.

Arize AI. 2024a. Arize Phoenix. https://arize.com/docs/phoenix.

Arize AI. 2024b. OpenInference Specification. https://arize-ai.github.io/openinference/spec/.

Bagga, A.; and Baldwin, B. 1998. Entity-Based Cross-Document Coreferencing Using the Vector Space Model. In *36th Annual Meeting of the Association for Computational Linguistics and 17th International Conference on Computational Linguistics, Volume 1*, 79–85.

Balusu, K. C. 2026. AgentTelemetry: A Fault Detection Benchmark and Toolkit for LLM Agent Observability. In *Proceedings of the 3rd ACM International Conference on AI-Powered Software (AIware '26)*.

Barke, S.; Goyal, A.; Khare, A.; Singh, A.; Nath, S.; and Bansal, C. 2026. AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. arXiv preprint arXiv:2602.02475.

Bouma, G. 2009. Normalized (Pointwise) Mutual Information in Collocation Extraction. In *From Form to Meaning: Processing Texts Automatically, Proceedings of the Biennial GSCL Conference 2009*, 31–40.

Bouzenia, I.; and Pradel, M. 2025. Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories. In *2025 IEEE/ACM 40th International Conference on Automated Software Engineering (ASE)*, 2846–2857.

Chen, M.; Wang, J.; Mu, F.; Wang, Y.; Liu, Z.; Feng, H.; and Wang, Q. 2026. Seeing the Whole Elephant: A Benchmark

for Failure Attribution in LLM-Based Multi-Agent Systems. arXiv preprint arXiv:2604.22708.

Datadog. 2025. LLM Observability. https://docs.datadoghq.com/llm_observability/.

Datadog. 2026. LLM Observability Patterns. https://docs.datadoghq.com/llm_observability/monitoring/patterns/.

Deng, X.; Gu, Y.; Zheng, B.; Chen, S.; Stevens, S.; Wang, B.; Sun, H.; and Su, Y. 2023. Mind2Web: Towards a Generalist Agent for the Web. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.

Dong, L.; Lu, Q.; and Zhu, L. 2024. AgentOps: Enabling Observability of LLM Agents. arXiv preprint arXiv:2411.05285.

Fan, S.; Ye, X.; Huo, Y.; Chen, Z.-Y.; Guo, Y.; Yang, S.; Yang, W.; Ye, S.; Chen, J.; Chen, H.; Cong, X.; and Lin, Y. 2026. AgentProcessBench: Diagnosing Step-Level Process Quality in Tool-Using Agents. In *Proceedings of KDD*.

Gao, J.; Sun, K.; Huang, J.-t.; Van Koevering, K.; Ji, S.; Huang, H.; Shi, W.; Lu, Z.; Xiao, Z.; Khashabi, D.; and Dredze, M. 2026. How to Interpret Agent Behavior. arXiv:2605.13625.

Google. 2024a. Perfetto Trace Processor. https://perfetto.dev/docs/analysis/trace-processor.

Google. 2024b. pprof. https://github.com/google/pprof.

Gregg, B. 2011. Flame Graphs. https://www.brendangregg.com/flamegraphs.html.

In, Y.; Tanjim, M.; Subramanian, J.; Kim, S.; Bhattacharya, U.; Kim, W.; Park, S.; Sarkhel, S.; and Park, C. 2026. Rethinking Failure Attribution in Multi-Agent Systems: A Multi-Perspective Benchmark and Evaluation. arXiv:2603.25001.

Laminar. 2025. Signals: Structured Event Extraction from Traces. https://docs.lmnr.ai/signals/introduction.

LangChain. 2024. LangSmith Observability. https://docs.langchain.com/langsmith/observability.

LangChain. 2026. LangSmith Insights. https://docs.langchain.com/langsmith/insights.

Langfuse. 2024. Langfuse Observability. https://langfuse.com/docs/observability/overview.

Lewis, D. D.; Yang, Y.; Rose, T. G.; and Li, F. 2004. RCV1: A New Benchmark Collection for Text Categorization Research. *Journal of Machine Learning Research*, 5: 361–397.

Li, H.; Yao, Y.; Zhu, L.; Feng, R.; Ye, H.; Wang, J.; He, Y.; Zou, P.; Zhang, L.; Lei, X.; Huang, H.; Deng, K.; Sun, M.; Zhang, Z.; Ye, H.; and Liu, J. 2026. CodeTracer: Towards Traceable Agent States. arXiv:2604.11641.

Liu, S.; Chen, Y.; Krishna, R.; Sinha, S.; Ganhotra, J.; and Jabbarvand, R. 2026a. Process-Centric Analysis of Agentic Software Systems. *Proceedings of the ACM on Programming Languages*, 10(OOPSLA1): 1961–1988.

Liu, Y.; Zhang, C.; Han, Z.; Liu, H.; Wang, Y.; Yu, Y.; Wang, X.; and Yin, Y. 2026b. TrajAD: Trajectory Anomaly Detection for Trustworthy LLM Agents. arXiv preprint arXiv:2602.06443.

Lù, X. H.; Kazemnejad, A.; Meade, N.; Patel, A.; Shin, D.; Zambrano, A.; Stańczak, K.; Shaw, P.; Pal, C. J.; and Reddy, S. 2025. AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories. arXiv preprint arXiv:2504.08942.

Ma, C.; Zhang, J.; Zhu, Z.; Yang, C.; Yang, Y.; Jin, Y.; Lan, Z.; Kong, L.; and He, J. 2024. AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents. In *Advances in Neural Information Processing Systems*, volume 37, 74325–74362.

Mace, J.; Roelke, R.; and Fonseca, R. 2015. Pivot Tracing: Dynamic Causal Monitoring for Distributed Systems. In *Proceedings of the 25th Symposium on Operating Systems Principles*, 378–393.

MacQueen, J. B. 1967. Some Methods for Classification and Analysis of Multivariate Observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, 281–297.

Mazaheri, P.; and Mazaheri, K. 2026. AgentAtlas: Beyond Outcome Leaderboards for LLM Agents. arXiv preprint arXiv:2605.20530.

McCallum, A.; and Nigam, K. 1998. A Comparison of Event Models for Naive Bayes Text Classification. In *AAAI-98 Workshop on Learning for Text Categorization*, 41–48.

Mulian, H.; Zeltyn, S.; Levy, I.; Galanti, L.; Yaeli, A.; and Shlomov, S. 2026. AgentFixer: From Failure Detection to Fix Recommendations in LLM Agentic Systems. In *Proceedings of the 1st International Workshop on Agentic Engineering (AGENT '26, co-located with ICSE)*.

Nian, J.; Chen, K.; Zhang, G.; Cao, Y.; and Jiang, Y. 2026. TraceGraph: Shared Decision Landscapes for Diagnosing and Improving Agent Trajectories. arXiv:2605.31308.

NVIDIA. 2026. Profiling and Performance Monitoring of NVIDIA NeMo Agent Toolkit Workflows. Official documentation.

OpenAI. 2025. Codex CLI: Lightweight Coding Agent That Runs in Your Terminal. https://github.com/openai/codex.

OpenTelemetry. 2024. OpenTelemetry GenAI Semantic Conventions. https://github.com/open-telemetry/semantic-conventions-genai.

OpenTelemetry. 2026. OpenTelemetry Profiles. Official specification.

Qwen Team. 2026. Qwen3.6-27B: Flagship-Level Coding in a 27B Dense Model. Official model card.

Robertson, S. 2008. A New Interpretation of Average Precision. In *Proceedings of the 31st Annual International ACM SIGIR conference on Research and Development in Information Retrieval*, 689–690.

Rosenberg, A.; and Hirschberg, J. 2007. V-Measure: A Conditional Entropy-Based External Cluster Evaluation Measure. In *Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning*, 410–420.

Ruokolainen, T.; Kohonen, O.; Sirts, K.; Grönroos, S.-A.; Kurimo, M.; and Virpioja, S. 2016. A Comparative Study of

Minimally Supervised Morphological Segmentation. *Computational Linguistics*, 42(1): 91–120.

Sajadi, A.; Nguyen, T.; Huynh, K.; Parra, E.; and Chatterjee, P. 2026. TraceView: Interactive Visualization of Agentic Program Repair Trajectories. arXiv:2606.22110.

Shu, R.; Chong, C. Y.; Zhou, X.; Peng, Y.; Wu, Z.; Han, X.; Zhuang, Z.; Yuan, G.; and Wang, Y. 2026. What Resolve Rate Hides: Trajectory Structure Diagnostics for Coding Agents. arXiv:2607.06184.

Wang, J.; Feng, Z.; Wu, J.; Li, R.; Xie, Q.; Ren, Y.; Zhu, H.; Han, X.; Meng, F.; Feng, J.; and Liu, J. 2026a. Where Do Deep-Research Agents Go Wrong? Span-Level Error Localization in Agent Trajectories. arXiv preprint arXiv:2606.02060.

Wang, J.; Hou, J.; Wang, F.; Jian, P.; Bao, C.; and Lv, Z. 2026b. HINTBench: Horizon-Agent Intrinsic Non-Attack Trajectory Benchmark. arXiv preprint arXiv:2604.13954.

Wang, R.; Jansen, P.; Côté, M.-A.; and Ammanabrolu, P. 2022. ScienceWorld: Is Your Agent Smarter than a 5th Grader? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 11279–11298.

Wang, Y.; Wu, W.; Wang, J.; and Wang, Q. 2026c. From Flat Logs to Causal Graphs: Hierarchical Failure Attribution for LLM-based Multi-Agent Systems. arXiv:2602.23701.

Wang, Y.; Zhang, J.; Cai, T.; Liu, Z.; Sun, Q.; Sun, Z.; Wu, Z.; Dong, M.; Zheng, M.; Yin, X.; and Zhu, Y. 2026d. From Agent Traces to Trust: A Survey of Evidence Tracing and Execution Provenance in LLM Agents. arXiv preprint arXiv:2606.04990.

WukLab. 2025. OSWorld-Human. https://github.com/WukLab/osworld-human.

Xie, T.; Zhang, D.; Chen, J.; Li, X.; Zhao, S.; Cao, R.; Hua, T. J.; Cheng, Z.; Shi, D.; Tong, J.; Lu, K.-W.; Garg, V.; Wang, Y.; Dai, Z.; Li, F.; Bisk, Y.; and Yu, T. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track*.

Yang, J.; Jimenez, C. E.; Wettig, A.; Lieret, K.; Yao, S.; Narasimhan, K.; and Press, O. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*.

Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E. P.; Zhang, H.; Gonzalez, J. E.; and Stoica, I. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.

Zheng, Y.; Hu, Y.; Yu, T.; and Quinn, A. 2025. AgentSight: System-Level Observability for AI Agents Using eBPF. In *Proceedings of the 4th Workshop on Practical Adoption Challenges of ML for Systems (PACMI '25, co-located with SOSP)*.

Zhong, Z.; Saxena, S.; and Raghunathan, A. 2026. Hodoscope: Unsupervised Monitoring for AI Misbehaviors. arXiv:2604.11072.